Agenti vocali e sessione madre

Come li abbiamo costruiti, cosa abbiamo sbagliato, e come rifarli su SAIO.

Questa guida descrive due cose che abbiamo costruito sopra SAIO in qualche mese di lavoro quotidiano:

1. **una flotta di agenti vocali** che rispondono al telefono per conto di attività commerciali (nel nostro caso locali che prendono ordini e prenotazioni), configurati uno per uno, corretti ogni giorno su richiesta dei clienti;
2. **la sessione madre**: una sessione Claude Code sempre aperta che coordina tutte le altre sessioni, parla con la persona responsabile su WhatsApp (testo e vocali), smista i guasti e tiene il filo quando il contesto si riempie.

Le due cose sono indipendenti: la Parte B e la Parte C valgono anche per chi non fa agenti vocali. Dove scriviamo numeri sono ordini di grandezza utili per tarare, non dati di un cliente.

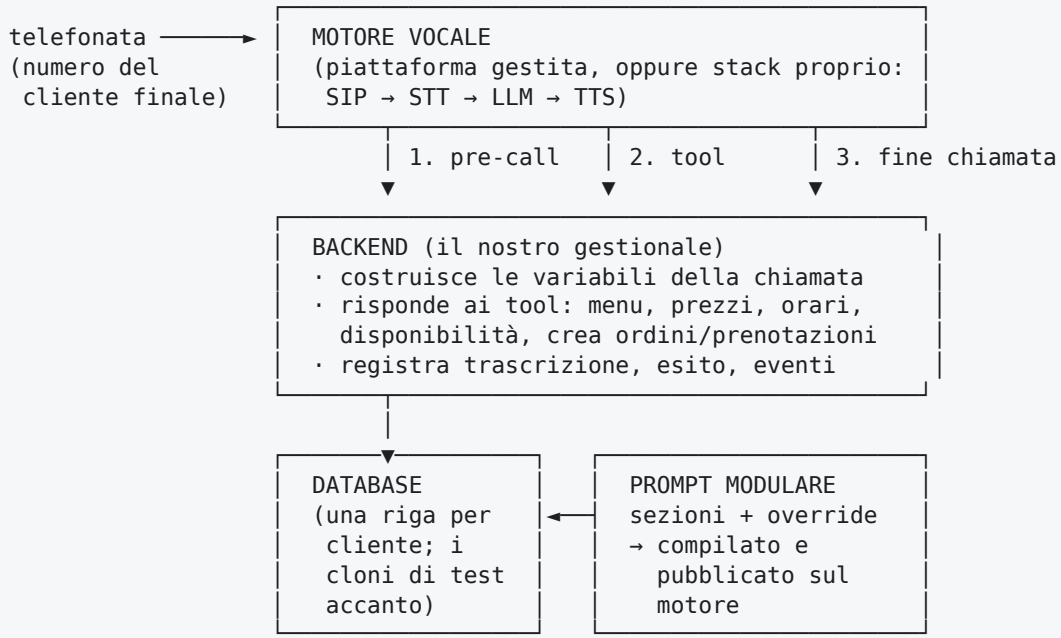
Indice

- **Parte A — Agenti vocali**
  - A1. L’architettura in una figura
  - A2. Il motore: piattaforma gestita o stack proprio
  - A3. Il pre-call: le variabili prima della risposta
  - A4. Il prompt modulare
  - A5. Gli strumenti (tool)
  - A6. Le leve: comportamenti per cliente
  - A7. Dopo la chiamata: post-call e analyzer
  - A8. I cloni di staging
  - A9. I giudici automatici
  - A10. Il collaudo
  - A11. Il ciclo delle revisioni dei clienti
  - A12. Le trappole
- **Parte B — La sessione madre**
  - B1. Cos’è e perché serve
  - B2. Come vede le altre sessioni
  - B3. Cosa fa e cosa non fa
  - B4. Il cancello: cosa resta alla persona
  - B5. WhatsApp in entrata
  - B6. WhatsApp in uscita
  - B7. Il brief del mattino e le urgenze
  - B8. I cron di sessione
  - B9. Memoria, handoff e il freno
  - B10. Le regole di comunicazione
  - B11. Gli errori che abbiamo fatto
- **Parte C — Rifarlo su SAIO**
  - C1. L’ordine in cui costruire
  - C2. Componenti minimi
  - C3. Pseudo-codice degli script chiave

- C4. Le istruzioni della madre (esempio di CLAUDE.md)
- C5. Checklist di sicurezza

Parte A — Agenti vocali

A1. L’architettura in una figura



Tre principi reggono tutto:

1. **Il motore parla, il backend decide.** L’LLM non fa conti, non sa gli orari, non inventa prezzi: chiede al backend con un tool e ripete la risposta. Tutto ciò che deve essere *giusto* (un totale, un orario libero, una via servita) sta nel codice, non nel prompt.
2. **Una sola fonte del prompt.** Il testo che l’agente riceve si compila da tabelle. Nessuno lo modifica a mano sul pannello del motore: la prossima pubblicazione lo sovrascriverebbe.
3. **Ogni cliente è una riga, non un fork.** Comportamenti diversi si ottengono con dati (override di sezioni, chiavi di configurazione, «leve»), mai con una copia del codice.

A2. Il motore: piattaforma gestita o stack proprio

Abbiamo usato entrambi, e li teniamo entrambi.

	Piattaforma gestita (tipo Retell)	**Stack proprio (tipo LiveKit + STT + LLM + TTS)**
Messa in produzione	ore	settimane
Latenza	buona, poco controllabile	controllabile, ma tutta a tuo carico
Costo al minuto	più alto	più basso, cresce coi volumi
Controllo sul turno	limitato (interruzioni, silenzi, attese)	totale
Osservabilità	quello che ti dà il pannello	tutto (tracce per turno, es. con Langfuse)
Guasti	li risolve il fornitore	li risolvi tu, anche di notte
Aggiornare l’agente	API: aggiorna LLM → pubblica versione	deploy del tuo worker

Come scegliamo. Si parte con la piattaforma gestita: è il modo più rapido di avere un cliente vero al telefono, e il prodotto si scopre lì. Lo stack proprio conviene quando (a) i volumi rendono il costo al minuto il problema principale, (b) servono cose che la piattaforma non permette (controllo fine dei turni, una lingua, un modello particolare), (c) hai qualcuno che può tenerlo in piedi la sera del sabato.

Il motore di riserva. Con lo stack proprio tieni sempre l'agente gestito pronto e allineato come ripiego: se il tuo worker non risponde, il numero ripiega sull'altro. Due regole:

- la riserva deve ricevere **gli stessi** prompt e strumenti, pubblicati dallo stesso processo — una riserva disallineata è peggio di nessuna riserva;
- un monitor deve dire *quando* si sta usando la riserva; altrimenti ci resti settimane senza saperlo.

Una trappola di entrambi: il motore di prova non è il motore di produzione. Se il clone di test risponde su un motore e il cliente vero su un altro, una prova verde sul clone non dice niente sul cliente. «Fatto» vuol dire provato sul motore che usa *quel* cliente.

A3. Il pre-call: le variabili prima della risposta

Prima che l'agente dica «pronto», il motore chiama un endpoint del backend (*inbound webhook* o equivalente) con il numero chiamato e quello chiamante. Il backend risponde con le **variabili dinamiche** della chiamata, che il prompt usa come segnaposto (`{{saluto}}`, `{{menu}}`, `{{orari_oggi}}`, …).

```
chiamata in arrivo
      │
      ▼
POST /precall { numero_chiamato, numero_chiamante }
      │
      ├─ quale cliente? (dal numero chiamato, o dall'agente)
      ├─ è aperto adesso? qual è il prossimo orario?
      ├─ chi chiama: cliente noto? ultimo ordine? numero nascosto?
      ├─ menu compatto per il prompt, regole di prezzo da dire
      └─ saluto giusto (buongiorno/buonasera, nome del locale)
      ▼
{ saluto, menu, orari, cliente_noto, istruzioni_prezzo, ... }
```

Cosa abbiamo imparato:

- **Il pre-call deve essere veloce e non deve mai fallire.** Se va in timeout l'agente parte senza variabili e dice cose senza senso. Cache in memoria per cliente, invalidata via eventi del DB (LISTEN/NOTIFY o equivalente) quando cambia il menu o la configurazione.
- **Tutto ciò che dipende dall'ora va calcolato nel fuso del cliente**, non del server.
- **Il saluto è una variabile, non testo fisso nel prompt.** Così cambia per fascia oraria e per cliente senza ripubblicare l'agente.
- **Decidi una volta dove vive la logica.** Noi l'avevamo prima in un workflow di automazione, poi l'abbiamo portata nel backend. Per un periodo coesistevano: due fonti, risultati diversi. Se la sposti, spegni la vecchia.

A4. Il prompt modulare

Il prompt dell'agente è **compilato** da sezioni. Ogni sezione ha un nome (saluto, raccolta ordine, consegna, pagamenti, chiusura, cosa non fare…) e un corpo che può esistere a tre livelli:

```
per ogni sezione:
   override del singolo cliente        (se c'è, vince)
      ↓ altrimenti
   corpo per tipo di attività          (es. solo asporto / asporto+consegna / tavoli)
      ↓ altrimenti
   corpo globale                       (vale per tutti)
      ↓ altrimenti
   sezione omessa
```

Più un livello di **flag**: una sezione può essere legata a una funzione («consegna a domicilio», «prenotazioni tavolo») e viene inclusa solo se quel cliente ha la funzione accesa.

Il compilatore produce un testo unico; un processo di **sync** lo pubblica sul motore (aggiorna l'LLM, pubblica una nuova versione dell'agente). I segnaposto `{{...}}` restano nel testo: li riempie il motore a ogni chiamata con le variabili del pre-call.

Regole che ci hanno salvato:

- **Niente fallback a testo libero.** Se il compilatore non produce nulla, l'agente non si pubblica e il sistema lo dice. Un vecchio prompt intero «di riserva» nel DB è la cosa che qualcuno modifica a mano un giorno e nessuno sa più quale versione sia viva.
- **Il prompt ha un tetto di lunghezza per sezione e per regola**, controllato in CI. I prompt crescono da soli (ogni revisione aggiunge una frase) e oltre una certa lunghezza l'agente segue peggio *tutte* le regole, non solo l'ultima.
- **Una regola del cliente si scrive corta e al presente**: «Le pizze baby non si fanno a domicilio». Niente storia («il cliente ci ha detto che…»), niente maiuscole urlate.
- **Prima di scrivere una regola chiediti se è una regola o un dato.** «La margherita costa 6 €» non va nel prompt: va nel menu, e il tool la legge. Le regole nel prompt servono per il comportamento, non per i fatti.
- **Rileva la deriva.** Un endpoint che confronta il prompt compilato con quello pubblicato sul motore. Prima o poi qualcuno toccherà il pannello a mano.

A5. Gli strumenti (tool)

Gli strumenti sono le chiamate HTTP che l'agente fa al backend durante la conversazione. I nostri, in astratto:

strumento	cosa fa
cerca nel menu	trova un piatto anche detto male, con sinonimi e varianti
totale ordine	prezza il carrello: quantità, varianti, aggiunte, composte, peso, consegna
disponibilità	quell'orario è libero? (cucina, forno, tavoli, fasce)
verifica indirizzo	la via è servita? quanto costa la consegna?
crea ordine / prenotazione	scrive, stampa la comanda, manda l'SMS di conferma
modifica / annulla	sull'ordine appena fatto
info locale	orari, chiusure, metodi di pagamento

Cose non ovvie:

- **La descrizione di uno strumento e dei suoi parametri è parte del prompt.** L'LLM sceglie *quale* strumento chiamare e *come* riempire i campi leggendo quelle descrizioni. Un

parametro descritto male produce dati sbagliati con la stessa fiducia di uno descritto bene. Esempio tipico: un peso detto dal cliente («mezzo chilo») finito nel campo «variante» invece che nel campo «grammi» → prezzo sbagliato in silenzio.

- **Ogni modifica alle definizioni degli strumenti richiede un sync verso il motore.** Il deploy del backend non basta: il motore ha la sua copia delle definizioni. Abbiamo un comando che ricostruisce e ripubblica gli strumenti di tutti gli agenti, e lo lanciamo dopo ogni deploy che li tocca.
- **Il backend deve essere tollerante sull'input e severo sull'output.** Accetta «due etti», «200 g», «0,2 kg»; risponde con una frase pronta da ripetere («Il totale è 18 euro e 50») e con i dati strutturati.
- **Lo strumento risponde con la frase da dire, non solo coi dati.** Così il testo che sente il cliente lo controlli tu, non l'LLM.
- **Gli strumenti lenti fanno silenzio.** Se una chiamata supera ~1 secondo, l'agente deve dire qualcosa («Un attimo.») prima di chiamarla, altrimenti il cliente chiede «c'è qualcuno?» o riattacca.
- **Autenticazione**: ogni chiamata porta l'identità dell'agente (quale cliente) e una chiave. Il backend deriva il cliente dall'identità dell'agente, mai da un parametro che l'LLM può riempire.

A6. Le leve: comportamenti per cliente

Con decine di clienti, ognuno chiede un'eccezione: «da noi la metà-e-metà costa come la più cara», «dopo le 22 niente consegne», «chiedi sempre quante pizze prima dell'orario». Se ognuna diventa un `if cliente == ...` nel codice, dopo un mese il codice è illeggibile.

La nostra soluzione si chiama **leva**: un comportamento del backend scritto una volta, in modo generale, e acceso per cliente da una chiave di configurazione.

```
codice:      if config(cliente).get("orario_chiede_quante_pizze"):
                 ... comportamento ...
config:      cliente A → { "orario_chiede_quante_pizze": true }
             cliente B → (assente: comportamento di serie)
```

Le regole delle leve:

1. **Nascono spente.** Una leva nuova non cambia niente a nessuno finché non la accendi per un cliente. Mai estendere a tutta la flotta un comportamento chiesto da uno.
2. **Ogni leva ha un catalogo**: nome, cosa fa, di che *natura* è (prezzo, orario, comunicazione…), dove compare nel pannello di configurazione. Una guardia in CI fallisce se una chiave letta dal codice non è nel catalogo. È fastidioso; è l'unica cosa che ci ha impedito di avere chiavi che nessuno sa a cosa servono.
3. **Si accende con una migration** dedicata al cliente, con il suo rollback e le sue prove (vedi A11), e si prova prima sul clone.
4. **Se una leva diventa utile a molti**, si discute se farla diventare il comportamento di serie. È una decisione esplicita, non uno scivolamento.

A7. Dopo la chiamata: post-call e analyzer

Alla fine di ogni chiamata il motore manda un evento con trascrizione, durata, esito. Il backend:

- salva la chiamata e collega ordini e prenotazioni creati durante la chiamata;
- lancia l'**analyzer**: un LLM legge la trascrizione e ricostruisce cosa è stato chiesto (piatti, orario, indirizzo, metodo di pagamento), per (a) recuperare un ordine che l'agente non è riuscito a creare, (b) segnalare differenze fra il detto e lo scritto.

Due lezioni dure:

- **L'analyzer può inventare.** Su una trascrizione vuota o tagliata, un LLM a cui chiedi «estrai l'ordine» un ordine lo trova. Serve una guardia esplicita: niente trascrizione → niente ordine, su *tutti* i motori, e niente ordini dopo una chiamata passata a una persona.
- **Chi vince fra agente e analyzer va deciso campo per campo.** Se il cliente ha confermato «sì» al riepilogo dell'agente, vale quello che ha detto l'agente; l'analyzer non deve correggerlo.

A8. I cloni di staging

Abbiamo un solo database. Ogni cliente vero ha un **clone**: una riga gemella con menu, orari e configurazione copiati, un suo agente sul motore, gli strumenti puntati al backend di staging, e un numero di prova.

```
cliente vero  ─── staging_of ──►  clone
(prod, riceve       │                 (agente suo, numero di prova,
 telefonate vere)   │                  backend di staging)
                    └── stesso schema, stesse migration, righe diverse
```

Perché funziona: una modifica di configurazione si scrive **una volta**, come migration parametrizzata sul cliente (`{{ID_CLIENTE}}`), si applica al clone, si prova, e poi la *stessa* migration si applica al cliente vero. Niente deploy di mezzo, niente copia a mano.

Perché è pericoloso:

- **«Solo staging» non esiste come permesso**: il clone sta nello stesso database del cliente vero. Chi può scrivere sul clone può scrivere su tutti; cambia solo la `WHERE`. Per le prove davvero isolate usiamo un ramo del database usa-e-getta (Neon lo permette in un secondo).
- **I puntelli di prova si cancellano subito.** Se per provare inventi una chiusura, un orario, un piatto sul clone, cancellalo nello stesso turno. Un pulsante «porta in produzione» che copia le differenze fra clone e vero non distingue una configurazione da un puntello: lo porterà sul cliente vero, giorni dopo. (L'abbiamo vissuto: una chiusura finta di un giorno è finita in produzione e l'agente ha detto «siamo chiusi» a clienti veri per mezz'ora.)

A9. I giudici automatici

Un **giudice** è un LLM che, dopo una chiamata, risponde a domande come «la chiamata è stata persa per colpa nostra?», «l'ordine registrato è uguale a quello detto?». Ne abbiamo diversi: sulle chiamate, sulle revisioni dichiarate pronte, sulla coerenza della configurazione.

Quello che abbiamo misurato confrontandoli col giudizio umano:

- **Un giudice ingenuo è d'accordo con l'umano solo in un terzo dei casi** sulle domande difficili («era persa? per colpa di chi?»), e inventa perdite che non esistono.
- Migliora molto se:
  1. **giudica in differita** (es. un'ora dopo): così vede se il cliente ha richiamato, se l'ordine è stato corretto, se lo staff ha sistemato — chi giudica 15 secondi dopo la chiamata non vede niente di tutto questo;
  2. **separa la gravità**: «chiamata persa» non è «imprecisione recuperata»;
  3. **ha un formato di risposta chiuso** (campi enumerati, non prosa);
  4. **è tarato su un insieme di casi giudicati da una persona**, con un insieme separato tenuto da parte solo per la verifica finale.
- Anche così, **un giudice è un setaccio, non un verdetto**. Lo usiamo per decidere cosa guardare, non per dire «va bene». La prova vera resta l'azione: il tool chiamato, l'item scritto,

il totale.

A10. Il collaudo

Il collaudo a voce non è un test unitario: è lento, costa, e la metà dei fallimenti viene dal cliente finto, non dall'agente. Il nostro ordine:

```
1. test automatici del backend        (il tool fa il conto giusto?)
2. prova del tool via API sul clone   (risposta giusta con i dati veri?)
3. chat testuale con l'LLM dell'agente (copia temporanea dell'LLM, stessi tool)
     ├─ una REGOLA di comportamento: almeno 3 ripetizioni verdi
     └─ un TOOL da chiamare:          almeno 8 ripetizioni verdi
4. chiamate registrate rigiocate       (il caso reale che aveva fallito)
5. 1-2 telefonate vere, alla fine, sul motore del cliente
```

E la regola su come si giudica: **sull'azione, non sulla frase.** Un agente che dice la cosa giusta ma non chiama lo strumento, o lo chiama coi parametri sbagliati, è un fallimento. Un agente che lo dice con parole diverse ma fa la cosa giusta, è un successo.

«Fatto» vuol dire la prova dell'**ultimo strato**, quello che usa il cliente finale: la chiamata vera, la comanda stampata, l'SMS arrivato. Test, build, deploy e tool via API sono strati sotto: verdi lì vuol dire «su staging, non provato». Questa distinzione, detta così, ci ha tolto metà dei «fatto» che non erano fatti.

Attenzione ai costi del collaudo automatico: un banco che telefona da solo in ciclo può fare centinaia di chiamate al giorno e finire un credito in un pomeriggio. Tetti per giorno e per cliente.

A11. Il ciclo delle revisioni dei clienti

Ogni giorno i clienti chiedono modifiche (il CSM le raccoglie nel gruppo WhatsApp del cliente). Le chiamiamo **revisioni**, a **round**: un round è un elenco di punti.

```
cliente scrive nel suo gruppo
      │
      ▼
CSM (o un'automazione che legge i gruppi) apre il ROUND nel CRM
      │       sezione: agenti | app | sito
      ▼
CODA: un classificatore decide chi lo lavora; chi lo prende lo RIVENDICA
      │       (UPDATE … WHERE preso_da IS NULL RETURNING — zero righe = è di un altro)
      ▼
SEMI-LOOP: apre una sessione Claude dedicata a quel round
      │       · verifica se è già fatto (spesso lo è)
      │       · scrive la migration del cliente  (dati/regole/leve)
      │         oppure il ramo di codice         (se serve codice)
      │       · applica sul CLONE, collauda (A10)
      │       · dichiara «pronta» con le prove
      ▼
QC umano su staging → «Porta in produzione» applica la stessa migration al cliente vero
      │       (con una guardia: non riapplicare una migration vecchia sopra una più nuova)
      ▼
i punti si SPUNTANO nel CRM (l'automazione può spuntare)
      ▼
la CHIUSURA del round resta UMANA
      (chiudere avvisa il CSM «puoi richiamare il cliente»: deve farlo chi ha
       controllato in produzione, non chi ha lavorato sul clone)
```

Dettagli che contano:

- **Ogni migration di cliente ha tre parti**: il cambio, le **prove** (SELECT che dimostrano l’effetto), il rollback. Il pulsante di produzione esegue le prove dopo aver applicato: se falliscono, si annulla.
- **Dopo una scrittura sul DB, invalida le cache** del backend con l’evento giusto e il payload giusto. Un payload sbagliato non dà errore: invalida la cosa sbagliata e la prova risulta verde.
- **Prima di lavorare un punto, guarda se è già fatto** (git, migration, una query). È il controllo che salta più spesso.
- **Ci si ferma per una cosa sola: un dato che ha solo il cliente** (un prezzo, un ingrediente, quali vie serve). Tutto il resto si decide e si va avanti.
- **Il ricettario.** Per ogni tipo di punto (regola di comportamento, voce di menu, capienza, stampa…) una ricetta scritta: dove si tocca, cosa si prova. Con qualche decina di ricette copri la grande maggioranza dei punti; il resto è codice.

A12. Le trappole

In astratto, quelle che ci sono costate di più:

1. **Il pannello del motore modificato a mano** → sovrascritto alla prossima pubblicazione, o peggio, non sovrascritto e divergente.
2. **Definizioni dei tool cambiate senza sync** → il backend nuovo, l’agente vecchio.
3. **Un comportamento per un cliente esteso a tutti** perché «tanto è giusto» → venti clienti che non l’avevano chiesto se ne accorgono il sabato sera.
4. **La configurazione di produzione diversa da quella di staging** (variabili d’ambiente, file letti dal processo): una funzione provata su staging è inerte in produzione, senza errori. Verifica sul processo vivo, non sul file che pensi venga letto.
5. **Un giudice automatico preso per verità** → decisioni su dati inventati.
6. **Due fonti della stessa logica** (automazione esterna + backend) → risultati diversi a seconda del ramo.
7. **Trasferimenti di chiamata non provati** → l’agente promette «le passo il locale» e la chiamata cade.
8. **Numeri nascosti, chiamanti stranieri, rumore di fondo**: vanno nei casi di prova fin dal primo giorno.
9. **La stampa**: la comanda deve dire esattamente ciò che ha detto il telefono. Se la stampa si ricostruisce i dati per conto suo, prima o poi diverge.

Parte B — La sessione madre

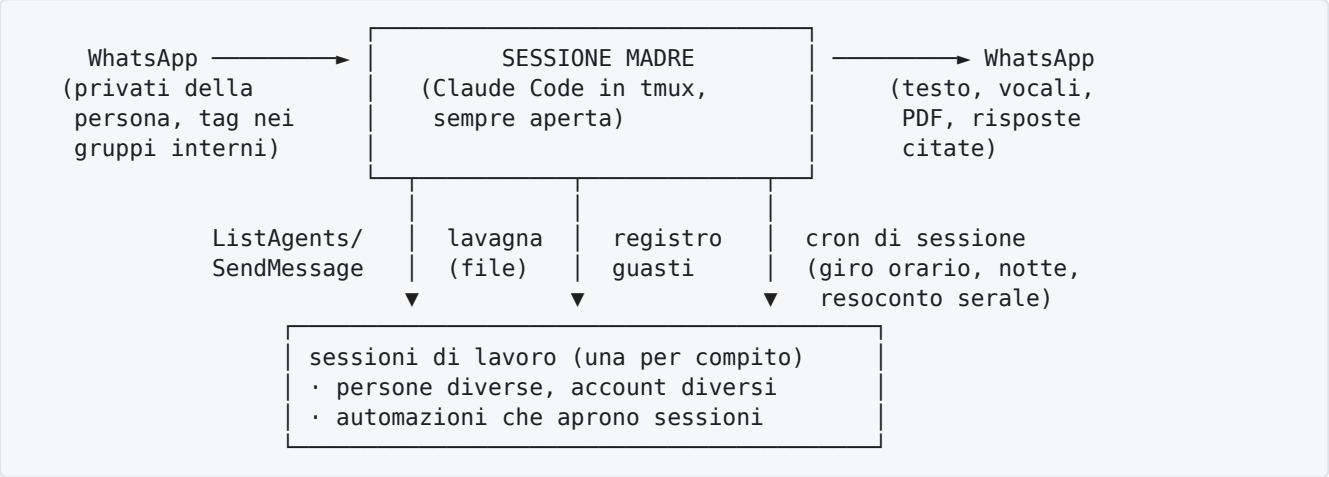
B1. Cos’è e perché serve

Quando lavori con molte sessioni Claude Code in parallelo (noi siamo arrivati a trenta-quaranta vive insieme, su una macchina condivisa, con più persone), succedono tre cose:

- le sessioni si bloccano su una domanda e **nessuno se ne accorge** per ore;
- due sessioni fanno **lo stesso lavoro**, o toccano lo stesso file;
- la persona responsabile riceve **troppe domande**, molte delle quali la sessione poteva risolvere da sola.

La **sessione madre** è una sessione Claude Code come le altre, aperta in tmux e lasciata viva, il cui unico compito è **coordinare**. Non scrive codice di prodotto. Il suo prodotto è:

- le altre sessioni non restano ferme;
- alla persona arrivano solo le decisioni che sono davvero sue, corte, sul canale che usa (il telefono);
- quello che si scopre ha sempre un proprietario.



B2. Come vede le altre sessioni

La madre ha quattro occhi:

1. **`ListAgents` / `SendMessage`** di Claude Code: l’elenco delle sessioni vive sulla macchina, il loro nome tmux, se sono occupate o ferme; e la possibilità di mandare un messaggio che arriva nella loro conversazione.
2. **`tmux capture-pane -p -t <sessione> | tail -30`**: cosa c’è sullo schermo adesso. È il modo di capire se una sessione è ferma su una domanda, su un limite dell’account, su un permesso. (Attenzione: le righe grigie in fondo a un pane fermo sono *suggerimenti* dell’interfaccia, non comandi digitati.)
3. **La lavagna**: un file condiviso dove ogni sessione dichiara all’inizio cosa sta per fare e quali file tocca. Il comando che la mostra affianca i file *dichiarati* a quelli *toccati davvero* (il `git status` di quel worktree, letto in quel momento). Il segnale sta nella differenza.
4. **Il registro dei guasti**: un file JSON con un id per guasto (`g101`, `g102`…), la descrizione, quante sessioni l’hanno visto, e **chi lo ha preso**. Un guasto senza proprietario è un guasto che nessuno ripara (ci è successo: un guasto citato da otto sessioni con otto diagnosi diverse, rimasto rotto due settimane).

Più uno script di **scansione** che, per tutte le sessioni, dice in una riga: viva o morta, ferma da quanto, ferma su cosa (domanda, limite, permesso, niente), ultimo prompt umano.

B3. Cosa fa e cosa non fa

Fa:

- **Smista i guasti.** Una sessione segnala qualcosa che non è suo → la madre lo registra, lo assegna alla sessione giusta (o ne apre una), e lo dice in una riga.
- **Gira gli ok della persona.** Quando la persona risponde «sì» a una domanda, la madre lo porta alla sessione che aspettava, **citando le parole testuali, l'ora e a quale domanda rispondeva**. Mai «la persona è d'accordo»: sempre «WhatsApp 09:58: "sì a tutti e due", in risposta a …».
- **Verifica prima di riferire.** Una sessione dice «sha abc123 in produzione»? La madre controlla che lo sha esista, che sia sul ramo giusto, che il deploy sia verde. Le sessioni a volte inventano gli sha.
- **Sblocca.** Una sessione ferma su un limite d'account → la sposta su un account libero e la riprende. Una sessione ferma su una domanda che la persona ha già deciso → risponde lei.
- **Chiude le sessioni finite** (salvando prima l'ultimo schermo in un file), e quelle di altri ferme da più di 48 ore.
- **Evita i doppioni.** Prima di dare un lavoro a una sessione, controlla che un'automazione non l'abbia già preso (nel nostro caso: la coda delle revisioni).

Non fa:

- **Non fa il lavoro delle altre sessioni.** Se scopre qualcosa che non è suo, lo dice al proprietario. Eseguire al posto di un'altra sessione può aggirare un permesso che quella sessione non aveva.
- **Non risponde ai messaggi che non lo richiedono.** «Ho finito», «sto lavorando su questi file»: si prende nota e basta. Niente «ok», niente ringraziamenti.
- **Non inventa decisioni della persona.** Se un ok è ambiguo («va bene così» detto a una domanda diversa), non è un ok.

B4. Il cancello: cosa resta alla persona

La regola che la persona ci ha dato, testuale: *le sessioni vanno avanti da sole; a me solo deploy di codice in produzione, scritture distruttive in produzione, messaggi nuovi ai clienti, soldi.*

Decide la madre (o la sessione)	Decide la persona
lavoro su un ramo, merge su staging, prove	deploy di **codice** in produzione
migration di configurazione già coperte dal mandato	scritture **distruttive o irreversibili** in produzione
aprire/assegnare un guasto, chiudere sessioni finite	**messaggi nuovi ai clienti** finali
rispondere alle domande tecniche delle sessioni	**soldi**: abbonamenti, macchine, contratti
messaggi nei gruppi **interni** del team	aprire o chiudere un **accesso**, ruotare chiavi

Ogni domanda alla persona si fa **una volta**, in forma chiusa, con la parola da rispondere:

```
*Dolci al chilo*: se il cliente dice «mezzo chilo», oggi il conto fa
pagare 1 kg. Correzione pronta, verde su staging, 8 pesi su 8 giusti.
→ «sì chilo»
```

Le domande aperte si tengono in un elenco; se la persona non risponde, si ricordano in una riga in fondo ai messaggi successivi, non si rifanno.

B5. WhatsApp in entrata

La persona usa il telefono, non il terminale. Quindi i suoi messaggi devono arrivare nella conversazione della madre.

```
telefono della persona
     │  scrive al numero dell'azienda (privato)
     │  oppure tagga il numero in un gruppo INTERNO del team
     ▼
istanza WhatsApp (Evolution API, multi-dispositivo)
     │
     ├──► flusso normale (automazioni, CRM, archivio)
     │      └─ uno SNIFFER scarta i privati della persona: non devono
     │         arrivare anche a un bot che risponderebbe al posto della madre
     │
     └──► CRON ogni minuto: lettore
            · legge da Evolution i messaggi nuovi della persona (privati)
              e i messaggi nei gruppi interni che taggano il numero o
              rispondono a un nostro messaggio
            · vocale → trascrizione (API veloce, mai un modello locale enorme)
            · immagine/documento → salvato su disco, si passa il percorso
            · ricostruisce a cosa risponde (il messaggio citato)
            · scrive nel PANE TMUX della madre:
                [WhatsApp da <persona> 09:58] (in risposta a «…») sì a tutti e due
                [WhatsApp gruppo <nome> | da <nome> 10:03] apri la revisione
                   [gruppo=<jid> id=<id>]
            · ricorda gli id già consegnati (niente doppioni), non salta
              avanti se una consegna fallisce (l'ordine conta)
```

Dettagli che fanno la differenza:

- **La citazione.** Quando la persona risponde a un messaggio specifico («sì» in risposta alla domanda sulla correzione X), la riga che arriva alla madre deve contenere l'inizio del messaggio citato. Senza, «sì» è ambiguo con tre domande aperte.
- **jid e id in coda alla riga**, per i gruppi: servono alla madre per rispondere *citando* esattamente quel messaggio, nello stesso gruppo.
- **La scrittura nel pane è un `tmux send-keys -l <testo>` seguito da Enter.** Semplice e robusto; la madre riceve il messaggio come se lo avesse scritto la persona. Per lo stesso motivo va trattato con le stesse cautele: **solo** la persona e i gruppi in lista bianca arrivano lì.
- **Al primo avvio si parte da adesso**: lo storico non si rigioca.

B6. WhatsApp in uscita

La madre risponde **sullo stesso canale** da cui la persona ha scritto: se scrive su WhatsApp, la risposta va su WhatsApp, non nel terminale.

cosa	come	quando
testo	`sendText`	il default
vocale	TTS → ogg/opus → `sendWhatsAppAudio`	risposte corte (sotto ~60 parole): stato, sì/no, «fatto»
documento	`sendMedia` con `mediatype: document`	rapporti, analisi: sempre PDF, mai un link a un file su disco
risposta in un gruppo	`sendText` con `quoted` sul messaggio	quando la persona tagga in un gruppo interno

Regole:

- **Lista bianca dei gruppi.** Uno script con l'elenco dei gruppi interni dove la madre può scrivere; un gruppo che non è in elenco viene rifiutato dal codice, non dal prompt.
- **I gruppi dei clienti non si toccano** dal canale della madre, salvo un flusso dedicato e approvato (noi ne abbiamo uno per alcuni clienti in prova, con le sue regole).
- **Il vocale si scrive come si parla**: niente sigle, percorsi, sha, simboli. Un vocale con «g-centonovantadue» non serve a nessuno.
- **Tetto giornaliero dei vocali** e un log di ogni invio (testo, destinatario, motore, costo stimato). La voce TTS ha una riserva locale (un modello piccolo che gira in CPU) se il servizio principale non risponde.
- **Le chiavi non passano mai a schermo**: gli script le ricevono dall'ambiente tramite un lanciatore che non le stampa (vedi C5).
- Ogni invio torna un id; tenerlo nel log serve a dimostrare che il messaggio è partito, e a citarlo dopo.

B7. Il brief del mattino e le urgenze

- **Di notte**, quello che la madre vuole dire alla persona al risveglio lo scrive in un file (`per-<persona>.md`), una riga per punto con la proposta.
- **Al mattino** uno script prepara un testo di qualche centinaio di parole dalle fonti (quel file, il resoconto della sera, i guasti aperti, cosa è andato in produzione) e un agente vocale **telefona** alla persona e glielo legge, raccogliendo le risposte. Se non risponde, lo stesso testo arriva come vocale. In ogni caso dopo arriva l'elenco scritto delle decisioni: è lì che si conferma. **Da una telefonata non parte niente che tocchi la produzione**: le decisioni dette a voce si confermano per scritto.
- **Urgenze**: una sentinella con regole chiuse (API di produzione giù da 5 minuti, allarme grave acceso da 10) che chiama la persona. Noi l'abbiamo tenuta **in ombra** una settimana (scrive «avrei chiamato» in un file e non chiama) prima di accenderla: serve a contare i falsi allarmi. Mai chiamate nelle ore di sonno, una chiamata per argomento ogni 24 ore.

B8. I cron di sessione

Claude Code permette di programmare prompt ricorrenti dentro una sessione. Sono **di sessione**: un riavvio li perde, quindi il loro testo esatto sta in un file (`cron-madre.md`) da cui si ricreano.

cron	quando	cosa fa
giro orario	ogni ora, in orario di lavoro	in silenzio: lavagna, guasti, carico e disco, ultimo deploy di staging, se il ramo principale entra pulito in staging. Scrive alla persona 3-6 righe **solo** se qualcosa è cambiato o serve una decisione; altrimenti una riga
giro di notte	ogni 20 minuti, di notte	scansiona le sessioni; quelle ferme sul limite dell'account le sposta su un account libero e le fa ripartire; risponde alle domande delle sessioni della persona decidendo come farebbe lei; mette in lista per il mattino le decisioni che restano sue. Nessun messaggio WhatsApp di notte
resoconto serale	una volta, la sera	tre sotto-agenti di sola lettura (git e deploy; sessioni dai transcript; revisioni e operatività) → un PDF di 2 pagine con 8 sezioni fisse (una riga, numeri, in produzione, incidenti, persone e carico, backlog, decisioni per la persona, domani) → un solo invio su WhatsApp
promemoria una tantum	data e ora fissate	«il 17 rileggi il file delle urgenze in ombra e chiedi se accenderle»

Più un **custode** fuori dalla sessione (crontab di sistema, ogni 2 minuti): guarda il pane della madre; se è fermo sul limite dell'account, la sposta su un altro account riprendendo la stessa conversazione, e le scrive di ricreare i cron. La madre non deve mai restare ferma per un limite.

B9. Memoria, handoff e il freno

Una sessione che lavora tutto il giorno riempie il contesto. Tre meccanismi:

1. La memoria persistente. Una cartella di file, **un fatto per file**, con un'intestazione (nome, descrizione di una riga, tipo: chi è la persona, una correzione ricevuta, un fatto di progetto, un riferimento) e un indice (`MEMORY.md`, una riga per file) caricato a ogni avvio. Ci si scrive solo quello che non sta già nel codice o nella storia git: le correzioni della persona e il loro perché, le decisioni prese, i canali. Esempi dei nostri, in astratto:

- «I deploy di codice in produzione solo con l'ok della persona citato testualmente» (+ perché);
- «Se la persona scrive da WhatsApp, la risposta va su WhatsApp»;
- «Messaggi di servizio certi ai gruppi dei clienti: li manda la madre dopo le 8, non li fa copiare ai CSM»;
- «Come funziona il canale WhatsApp → madre: script, cron, nodo dell'automazione».

2. L'handoff. Quando il contesto si avvicina al limite, un hook chiede alla sessione di scrivere un file di passaggio con sezioni fisse:

```
1. OBIETTIVO
2. GIÀ FATTO E VERIFICATO        (con le prove: sha, id dei messaggi, query)
3. NON RIFARE                    (messaggi già mandati, analisi già fatte)
4. IN VOLO                       (agenti in background, sessioni che riferiranno, cron)
5. COSA ASPETTO DA TE            (le domande aperte alla persona, testuali)
6. PROSSIMO PASSO
7. DA RILEGGERE PRIMA DI SCRIVERE (le fonti dello stato vero)
8. TRAPPOLE INCONTRATE
```

Poi il contesto si svuota (`/clear`) e la sessione riparte leggendo il file. La sezione «NON RIFARE» è la più importante: senza, la sessione nuova rimanda ai clienti messaggi già mandati.

3. Il freno. Se nell'handoff la sezione «COSA ASPETTO DA TE» non è vuota, un hook blocca gli strumenti di scrittura finché la persona non scrive qualcosa. La sessione ripete la domanda aperta in tre righe e si ferma. Il motivo: una sessione che riparte dopo un riassunto tende a scambiare la propria domanda per una risposta («il report dice che ha detto sì») e ad agire.

B10. Le regole di comunicazione

Le abbiamo misurate sulle conversazioni vere, e sono diventate istruzioni:

- **Corto.** La risposta tipica utile è di poche righe. Quelle che fanno esplodere chi legge sono sopra i duemila caratteri, e più della metà stanno *raccontando una verifica*.
- **Rispondi alla domanda che ti è stata fatta.** A «è pronto?» si risponde sì o no, più una riga sul perché. La prova si tiene pronta se la chiedono.
- **«Fatto» solo con la prova dell'ultimo strato** (A10).
- **Una domanda si fa una volta.** Se non ha avuto risposta, una riga in fondo ai messaggi dopo.
- **Se puoi lavorare, non chiedi.** Tre prove: posso farlo senza rischi (staging, un ramo, un'analisi)? la risposta la trovo da solo (DB, git, memorie, documenti)? se resto fermo, cosa si ferma? Se la risposta all'ultima è «niente», non è una domanda: è una riga di rapporto.
- **Quando sbagli, dillo nella riga dopo.** «Hai ragione, ho sbagliato prima: …». Niente giustificazioni.
- **Niente gergo con le persone non tecniche** del team: con loro si parla di clienti e di cose che succedono, non di file e sha.

B11. Gli errori che abbiamo fatto

Ognuno è diventato una regola. In astratto:

errore	regola nata
nei mandati alle sessioni la madre scriveva «poi in produzione coi rilasci normali»: quattro deploy di codice sono partiti senza che la persona li avesse visti	il codice va in produzione solo con l’ok della persona **citato testualmente**; nei mandati si scrive sempre «fermati a staging, sha e prova alla madre»
un avviso al team diceva «X ha risposto e lo passa a voi» senza dire cosa restava da fare	ogni passaggio a una persona dice **cosa resta a lei**, in una riga
la madre ha dato un lavoro a una sessione mentre un’automazione l’aveva già preso	prima di assegnare, si guarda la **coda** dell’automazione
un’analisi ha cercato una parola («QR») e ha concluso «a 25 clienti non è mai stato mandato»: il messaggio c’era, era un video senza quella parola	«zero risultati» non è una risposta: due-tre formulazioni, e i media senza testo
una sessione ha riferito uno sha che non esisteva	la madre verifica sha, ramo e deploy prima di riferire «in produzione»
un comportamento chiesto da un cliente è stato esteso a tutti perché la persona aveva detto «sempre»	«sempre» si riferisce a quel cliente; per estendere serve una conferma esplicita
i timer di sistema che dovevano mandare messaggi fallivano perché il lanciatore delle chiavi non riconosceva l’identità	se un invio automatico fallisce, la madre lo manda a mano e lo segnala; un guasto «noto» ha un proprietario
un monitor segnalava staging rosso, ma era un run vecchio già superato	prima di agire su un allarme, guarda data e sha del run

Parte C — Rifarlo su SAIO

C1. L’ordine in cui costruire

1. **Le sessioni con nome stabile.** Ogni sessione di lavoro in tmux con un nome che dice di chi è e cosa fa (`<persona>-<compito>`). Un worktree git per sessione. È la base di tutto il resto.
2. **La lavagna e il registro dei guasti.** Due file JSON e un piccolo comando (`lavagna apri/chi/chiudi`, `guasto "<testo>"`, `guasto prendo/chiudi <id>`). Le sessioni li usano per istruzione (nel CLAUDE.md globale).
3. **La sessione madre in tmux** con il suo CLAUDE.md (C4) e la memoria persistente.
4. **Il canale in uscita** (testo e PDF su WhatsApp alla persona). Prima l’uscita: così la madre è utile anche se la persona continua a rispondere dal terminale.
5. **Il canale in entrata** (cron → pane). Qui la madre diventa «raggiungibile dal telefono».
6. **I cron di sessione** (giro orario, notte, resoconto) e il file da cui si ricreano.
7. **Il custode** (cambio account sul limite) e **l’handoff con il freno**.
8. **Vocali, brief del mattino, gruppi interni**: per ultimi, quando il resto è stabile.

C2. Componenti minimi

```
saio-madre/
├── CLAUDE.md                istruzioni della madre (C4)
├── cron-madre.md            testo esatto dei cron, per ricrearli
├── persona.json             numero/LID della persona, gruppi interni (lista bianca)
├── whatsapp_a_madre.py      lettore: Evolution → pane tmux (cron ogni minuto)
├── manda.py                 testo / documento alla persona
├── rispondi_gruppo.py       risposta citata in un gruppo interno
├── voce.py                  testo → TTS → vocale (tetto giornaliero, log)
├── scansiona_sessioni.sh    una riga per sessione: viva, ferma da, ferma su
├── custode.py               limite account → cambio account → riprendi
├── per-persona.md           cosa dire al risveglio
└── log/                     un jsonl per ogni canale in uscita
```

Servizi esterni: un’istanza WhatsApp (Evolution API o equivalente), un servizio di trascrizione veloce, un TTS (con una riserva locale), un posto per i segreti con un lanciatore che li mette nell’ambiente del comando senza stamparli.

C3. Pseudo-codice degli script chiave

Lettore WhatsApp → pane (cron ogni minuto):

```
STATO = "whatsapp_a_madre.json"      # {"dal": ts, "consegnati": [id, ...]}
PANE  = "madre"                       # sessione tmux della madre

def main():
    stato = carica(STATO)
    if "dal" not in stato:                      # primo giro: si parte da adesso
        salva(STATO, {"dal": now(), "consegnati": []}); return
    msgs = privati_della_persona() + tag_nei_gruppi_interni()
    nuovi = [m for m in sorted(msgs, key=ts)
             if ts(m) >= stato["dal"] and m.id not in stato["consegnati"]]
    for m in nuovi:
        try:
            tmux_scrivi(PANE, riga(m))          # send-keys -l <testo>; Enter
        except Exception:
            log("riprovo al prossimo giro"); break   # non saltare avanti
        stato["consegnati"] = (stato["consegnati"] + [m.id])[-500:]
        salva(STATO, stato)

def riga(m):
    testo = trascrivi(m) if m.vocale else salva_media(m) if m.media else m.testo
    citato = f' (in risposta a «{taglia(m.citato, 80)}»)' if m.citato else ""
    if m.gruppo:
        return (f"[WhatsApp gruppo {nome(m.gruppo)} | da {m.autore} {ora(m)}]{citato} "
                f"{testo} [gruppo={m.gruppo} id={m.id}]")
    return f"[WhatsApp da {PERSONA} {ora(m)}]{citato} {testo}"

def tag_nei_gruppi_interni():
    return [m for g in GRUPPI_INTERNI for m in ultimi(g)
            if tagga_noi(m) or risponde_a_noi(m)]
```

Risponditore nei gruppi:

```
def rispondi(jid, testo, cita=None):
    if jid not in GRUPPI_INTERNI:
        sys.exit(2)                             # il rifiuto sta nel codice
    corpo = {"number": jid, "text": testo}
    if cita:
        corpo["quoted"] = {"key": {"id": cita, "remoteJid": jid}}
    r = evolution_post("/message/sendText/<istanza>", corpo)
    log_jsonl({"jid": jid, "testo": testo, "id": r.key.id, "cita": cita})
```

Vocale:

```
def voce(testo, destinatario, cita=None):
    if parole(testo) > 60: sys.exit(2)          # oltre: testo o PDF
    if invii_di_oggi() >= TETTO: sys.exit(3)
    audio = tts_principale(testo) or tts_locale(testo)   # riserva in CPU
    ogg = a_opus(audio)
    r = evolution_post("/message/sendWhatsAppAudio/<istanza>",
                       {"number": destinatario, "audio": b64(ogg), "quoted": ...})
    log_jsonl({..., "motore": ..., "costo_stimato": ...})
```

Custode (crontab di sistema, ogni 2 minuti):

```
schermo = tmux_capture(PANE)
if fermo_sul_limite(schermo) or (account_quasi_pieno() and sessione_ferma(schermo)):
    if minuti_dall_ultimo_cambio() > PAUSA:
        sposta_su_account_libero(PANE)          # stessa conversazione, --resume
        aspetta_pronta(PANE)
        tmux_scrivi(PANE, "Ricrea i cron da cron-madre.md e continua.")
```

Un cron di sessione (dentro Claude Code; testo esatto in `cron-madre.md`):

```
13 8-23 * * *   Giro di coordinamento. In silenzio: lavagna, guasti, carico,
                ultimo deploy di staging. Messaggio a una sessione SOLO per una
                collisione vera o per sbloccarla. Alla persona 3-6 righe solo se
                è cambiato qualcosa o serve una sua decisione; altrimenti una riga.
```

C4. Le istruzioni della madre (esempio di CLAUDE.md)

```
# Sessione madre di <persona>

Sei la sessione di coordinamento. Non scrivi codice di prodotto.

## Cosa resta a <persona>
Deploy di codice in produzione, scritture distruttive in produzione,
messaggi nuovi ai clienti, soldi, accessi. Tutto il resto lo decidi tu
come lo deciderebbe lei (vedi le memorie).

## Come chiedi
Una domanda per decisione, in forma chiusa, con la parola da rispondere
(→ «sì X»). Una volta sola. L'elenco delle domande aperte lo tieni tu.

## Come giri un ok
Alla sessione interessata, citando: canale, ora, parole testuali, a quale
domanda rispondeva, lo sha. «Fermati a staging» se l'ok non c'è.

## Canali
- Messaggi che iniziano con [WhatsApp da <persona> …]: rispondi su WhatsApp
  (manda.py; voce.py se sotto 60 parole).
- [WhatsApp gruppo … [gruppo=<jid> id=<id>]]: rispondi in quel gruppo
  citando l'id (rispondi_gruppo.py / voce.py --cita).
- Rapporti: PDF, mai un percorso di file.

## Le altre sessioni
- A un messaggio che non chiede niente non si risponde.
- Prima di riferire «in produzione»: sha esiste, è sul ramo, deploy verde.
- Un guasto che non ha proprietario lo registri e lo assegni.
- Non fai il lavoro di un'altra sessione.
- Prima di assegnare un lavoro, guarda se un'automazione l'ha già preso.

## «Fatto»
Solo con la prova dell'ultimo strato (quello che vede il cliente).

## Contesto
Quando ti viene chiesto l'handoff: le 8 sezioni, NON RIFARE compresa.
```

C5. Checklist di sicurezza

- ☐ **I segreti non passano mai a schermo.** Un lanciatore (`run --con NOME -- comando`) mette la chiave nell’ambiente del solo comando e ne oscura il valore anche se il comando lo stampa. Quello che compare a schermo finisce nei transcript, e i transcript si leggono.
- ☐ **Lista bianca dei gruppi** nel codice degli script di invio; un gruppo non in elenco viene rifiutato con un codice di uscita.
- ☐ **Solo la persona e i gruppi interni arrivano nel pane.** Il lettore filtra per numero/LID del mittente; qualunque altro messaggio non diventa mai un prompt della madre.
- ☐ **I privati della persona escono dal flusso dei bot** (nessuna risposta automatica che parli al posto della madre).
- ☐ **Tetto giornaliero** su vocali e telefonate; log con costo stimato per ogni invio.
- ☐ **Niente scritture nei gruppi dei clienti senza una conferma umana**, salvo un flusso dedicato con le sue regole.

- ☐ **Nessuna decisione da una telefonata**: ciò che tocca la produzione si conferma per scritto.
- ☐ **Orari di silenzio**: niente messaggi e niente chiamate di notte, salvo le urgenze con regole chiuse (e prima una settimana in ombra).
- ☐ **Il database è in sola lettura di default** per le sessioni; le scritture di prova in una transazione annullata o su un ramo del DB usa-e-getta.
- ☐ **Backup prima di toccare un'automazione in produzione** (es. un nodo di workflow), e una riga che dica cosa è cambiato: quei cambi non compaiono in nessun `git diff`.

Guida scritta l'11/10/2026. I meccanismi descritti sono in uso ogni giorno; i nomi di script e file sono indicativi, da adattare alla propria installazione di SAIO.